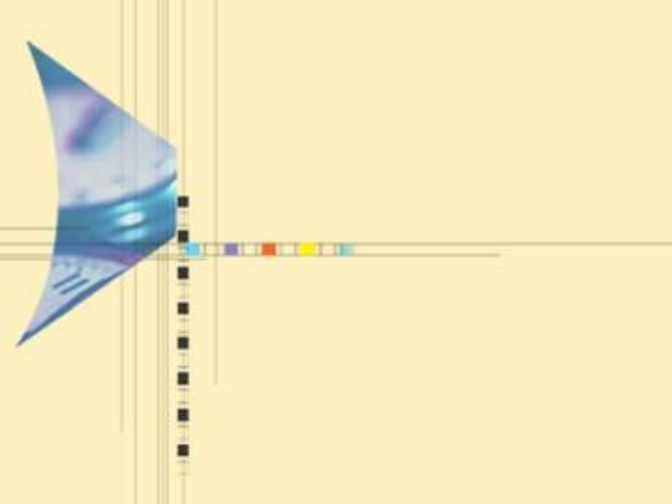




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

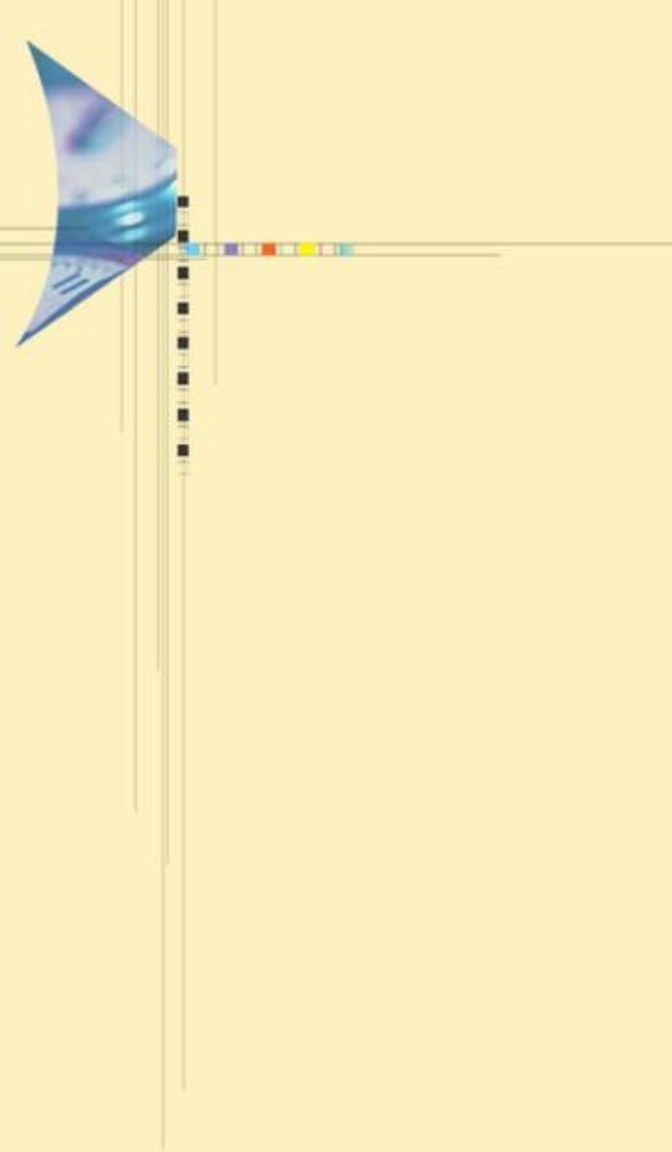
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/18

武汉大学计算机学院

武汉大学
Wuhan University





第八讲

回溯法

—An Efficient Searching Algorithm



主要内容

8.1 引言

8.2 回溯法简单例

8.3 一般回溯法

8.4 回溯法的典型应用

克服困难性的方法

- 已知很多实际问题尚无有效的算法，并且关于这些问题的已知算法，即便对于中等规模的实例，需要的时间量也非常大（年甚至实际为单位）
- 三种应对这类困难性的方法：
 - 方法一：适用于那些在平均情况下能显示出良好的时间复杂性，但在最坏情况下很难得到多项式解法的问题。这套方法基于对被研究的问题实例所导出的隐状态空间进行有条理的检查，在探查状态空间的过程中，会对搜索空间进行裁剪；
 - 方法二：基于精确度的概率概念。这些解的核心是对完成一个任务进行简单的决策制定或测试（成功或失败二选一），而不涉及其他选项。经过多次迭代后构造出解或者将解的置信度增加到所期望的程度；
 - 方法三：对于得到渐近解非常有用。人们愿意在解的质量上妥协以换取更快的（多项式时间）求解过程。仅有某些困难问题类允许这种多项式近似，且其中仅有极少一部分才提供了多项式时间的系列解，其中多项式的阶是精确性的函数。

困难性的应对方法之一

- 在许多真实世界的问题中，像大多数NP难问题一样，通过穷尽搜索数量巨大但有限多个可能性，可以获得一个解。
- 事实上对于所有这些问题都不存在用穷尽搜索之外的方法来解决问题的算法。
- 怎么样开发出系统化的搜索技术？怎样能够将搜索空间减少到尽可能的小？
 - 回溯法
 - 分支限界法

回溯法简介

- 回溯法 (backtracking) 是一种系统地搜索问题解的方法。为实现回溯, 首先需要定义一个解空间 (solution space), 然后以易于搜索的方式组织解空间, 最后用深度优先的方法搜索解空间, 以获得问题的解。
- 一般而言, 回溯法求解的三个步骤:
 - (1) 定义一个解空间, 它包含问题的解
 - (2) 用易于搜索的方式组织解空间
 - (3) 深度优先搜索解空间, 获得问题的解。

(1) 定义解空间

- 问题的解一般可表示为向量（也称元组）形式 $X=(x_1, x_2, \dots, x_n)$ ，其中分量 x_i 的取值范围为有穷且可枚举的集合 S_i 。
- 每个 x_i 的所有可能取值的组合构成的集合，称为问题的解空间。其中每一个取值组合对应问题实例的一个可能解。

例1

- 对0/1背包问题，对 n 个物品，定义解的形式为 n 元组 $X=(x_1, x_2, \dots, x_n)$ ，其中每个分量 x_i 的取值范围 $S_i=\{0,1\}$ 。所有0和1构成的 n 元组构成了问题的解空间，由 2^n 种可能的解组成。
- 例如，当 $n=3$ 时，0/1背包问题的解空间如下：
 $\{(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)\}$

组合问题——问题包含元素所有可能的非空子集，看哪一个非空子集满足要求

例2

- 对货郎担(旅行商)问题, 设城市个数为 n , 定义解的形式为元组 $X=(x_1, x_2, \dots, x_n)$, 其中每个分量 x_i 的取值范围 $S_i = \{1, 2, \dots, n\}$, 包含有 n^n 种可能的解。如果考虑到每个城市恰好访问1次的周游路线约束条件, 解空间压缩为包含 $n!$ 种可能的解。
- 例如, 当 $n=3$ 时, $S=\{1, 2, 3\}$, 货郎担问题的解空间是:
 $\{(1, 1, 1), (1, 1, 2), (1, 1, 3), (1, 2, 1), (1, 2, 2), (1, 2, 3), \dots, (3, 3, 1), (3, 3, 2), (3, 3, 3)\}$

考虑到约束条件, 货郎担问题的解空间压缩为:

$\{(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)\}$

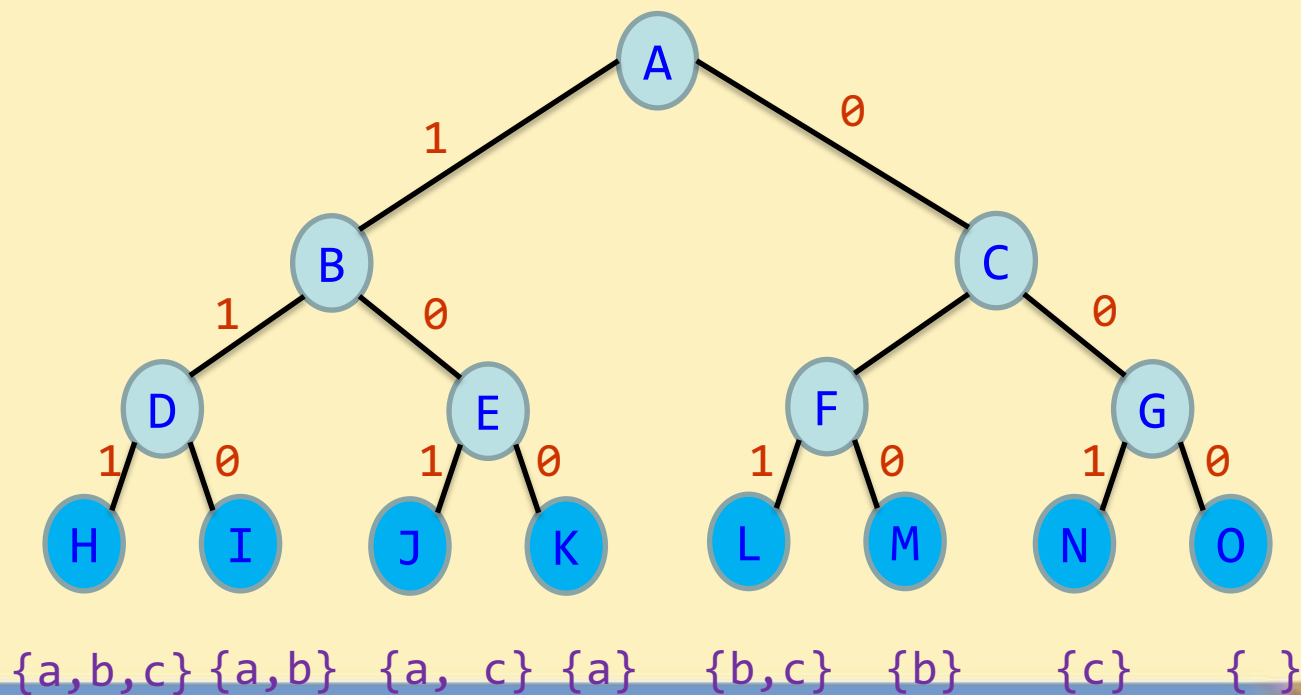
排列问题——问题包含元素必须出现, 看哪一个排列满足要求

(2) 解空间的树结构

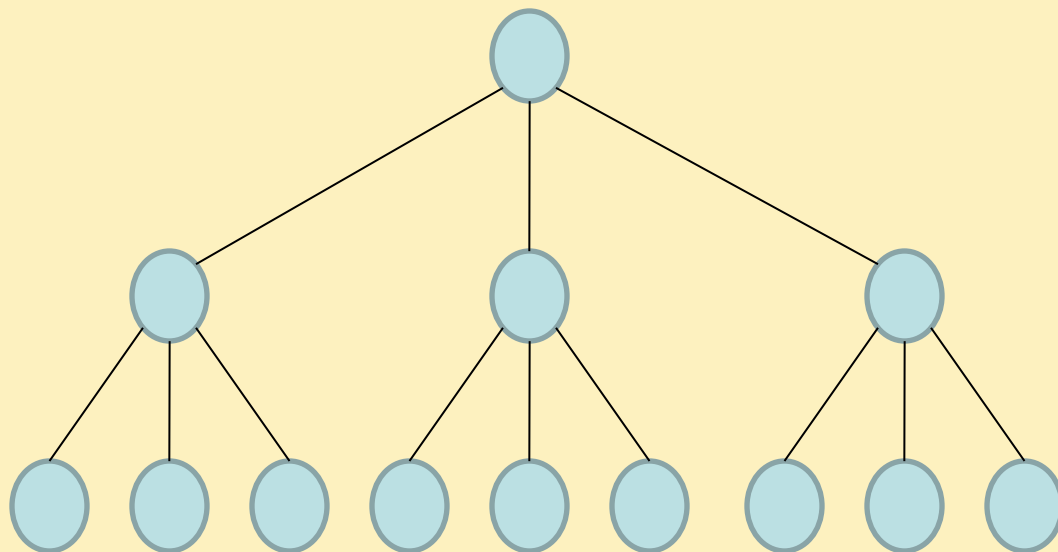
- 用一棵树来组织解空间。树中的每个结点确定所求问题的一个**问题状态**。
- 由根结点到其它结点的所有路径确定了这个问题的**状态空间**。
- **解状态**是这样一些问题状态 S ，对于这些问题状态，由根到 S 的那条路径确定了这解空间中的一个元组。
- **答案状态**是这样的一些解状态 S ，对于这些解状态而言，由根到 S 的这条路径确定了这问题的一个满足条件的解。
- 组织解空间的树结构称为**状态空间树或解空间树**。

解空间树通常有两种类型：

- **子集树**：当所给的问题是从 n 个元素的集合 S 中找出满足某种性质的**子集**时，相应的解空间树称为子集树。



- **排列树**：当所给的问题是确定 n 个元素满足某种性质的排列时，相应的解空间树称为**排列树**。

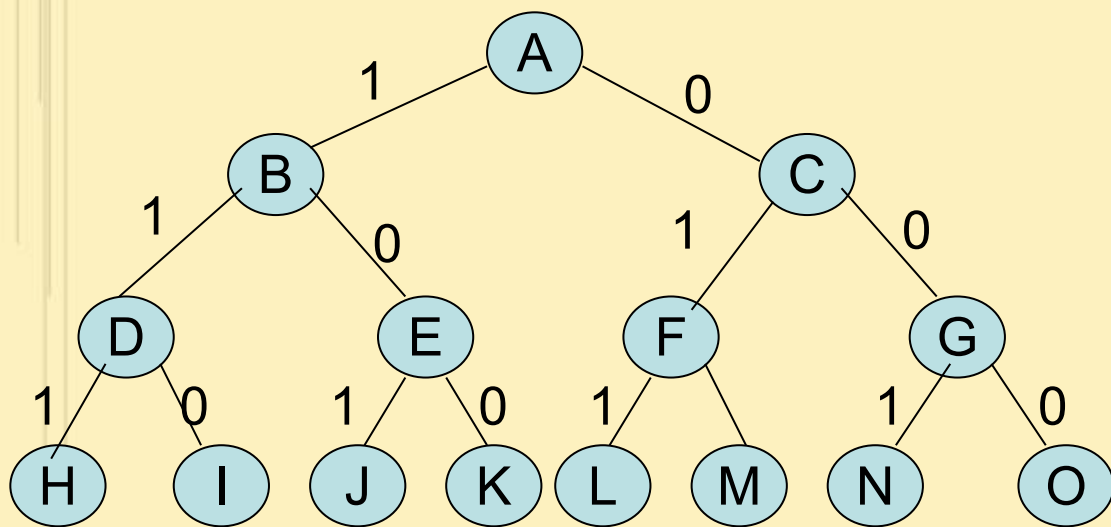


0/1背包问题的状态空间树

例：n=3, 0/1背包问题的状态空间树：

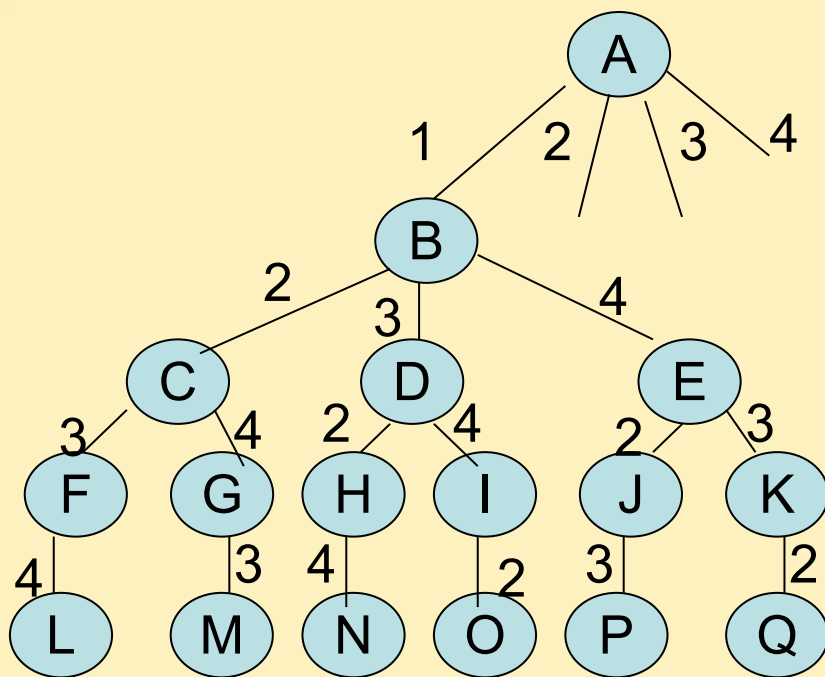
----子集树

解空间大小： 2^3



旅行商问题的状态空间树

例： $n=4$ ，旅行商问题的状态空间树（部分）：



----排列树

解空间大小: $4!$

状态空间树=>问题解

- 对于任何一个问题,一旦设想出一种状态空间树,就可以系统地生成问题状态,接着确定这些问题状态中的哪些状态是解状态,最后确定哪些解状态是答案状态,从而将问题解出。
- 问题状态的生成:从根结点开始生成树的其它结点。
- **活结点**:如果已生成一个结点而它的儿子结点还没有全部生成,则这个结点叫做活结点;当前正在生成其儿子结点的活结点叫E-结点(**正在扩展的结点**)。
- **死结点**:不再进一步扩展或者其儿子结点已经全部生成的已生成结点是死结点。

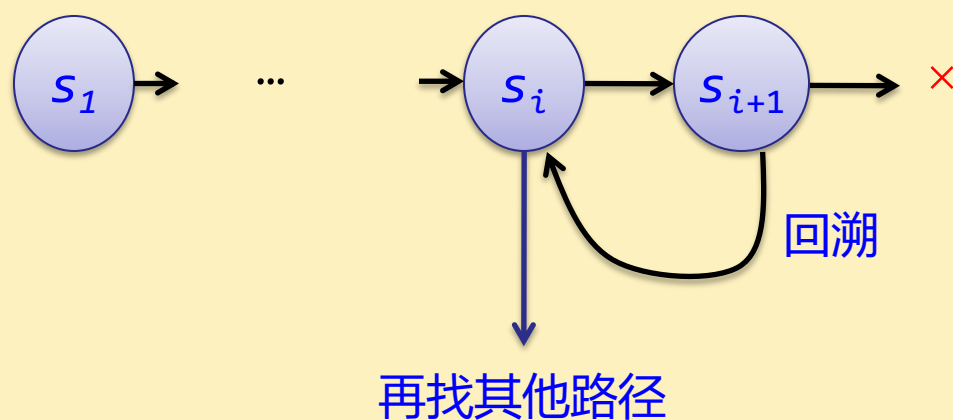
(3) 深度优先搜索

- 解空间组织称状态空间树后，回溯法就从开始结点（状态空间树的根结点）出发，以深度优先的方式搜索整个解空间。
- 这个开始结点就成为一个活结点，同时也成为当前的扩展结点。
- 在当前的扩展结点处，搜索向纵深方向移至一个新结点。这个新结点就成为一个新的活结点，并且成为当前的扩展结点。
- 如果在当前的扩展结点处不能再向纵深方向移动，则当前的扩展结点就成为死结点。此时应往回移动（回溯）至最近一个活结点处，并使这个活结点成为当前扩展结点。

回溯法即以这种工作方式递归地在解空间中搜索，直至找到要求的解或解空间中已无活结点时为止。

可见，回溯法实际上在搜索的同时逐步生成解空间树（实际只需要一部分）。也就是说，对于实际问题我们不必要搜索整个解空间。

当从状态 s_i 搜索到状态 s_{i+1} 后，如果 s_{i+1} 变为死结点，则从状态 s_{i+1} 回退到 s_i ，再从 s_i 找其他可能的路径，所以回溯法体现出走不通就退回再走的思路。



- 若用回溯法求问题的所有解时，需要回溯到根结点，且根结点的所有可行的子树都要已被搜索完才结束。
- 而若使用回溯法求任一个解时，只要搜索到问题的一个解就可以结束。

(3) 深度优先搜索

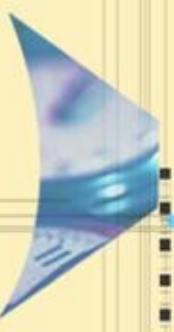
- 为了使搜索更加有效，我们常常在搜索过程中加一些判断以决定搜索是否该终止或改变路线。
- 通常采用两种策略来避免无效的搜索，提高回溯法的搜索效率。
 - (1) 使用约束函数在扩展顶点处剪去不满足约束的子树；
 - (2) 如果是优化问题，用限界函数剪去不能得到最优解的子树。

这两种函数统称为剪枝函数。

举例：

8—皇后问题：要求在一个 8×8 的棋盘上放8个皇后，且使得每两个之间都不能互相“攻击”，即都不出现在同一行、同一列及同一条斜对角线上。

- 解的表示：假定第 i 个皇后放在第 i 行，用8—元组 $(x_1, x_2, \dots, x_8)$ 表示解，其中 x_i 表示放置皇后 i 的列号。
- 约束条件：
 - 显示约束： $S_i = (1, 2, 3, \dots, 8), 1 \leq i \leq 8$ （规定 x_i 的取值集合）
 - 隐示约束：没有两个 $x_i (1 \leq i \leq 8)$ 可以相同（在不同的列上），且没有两个皇后在同一斜对角线上。（规定 x_i 之间的彼此相关情况）



主要内容

8.1 引言

8.2 回溯法简单例

8.3 一般回溯法

8.4 回溯法的典型应用

8.2.1 图的三着色问题

问题描述

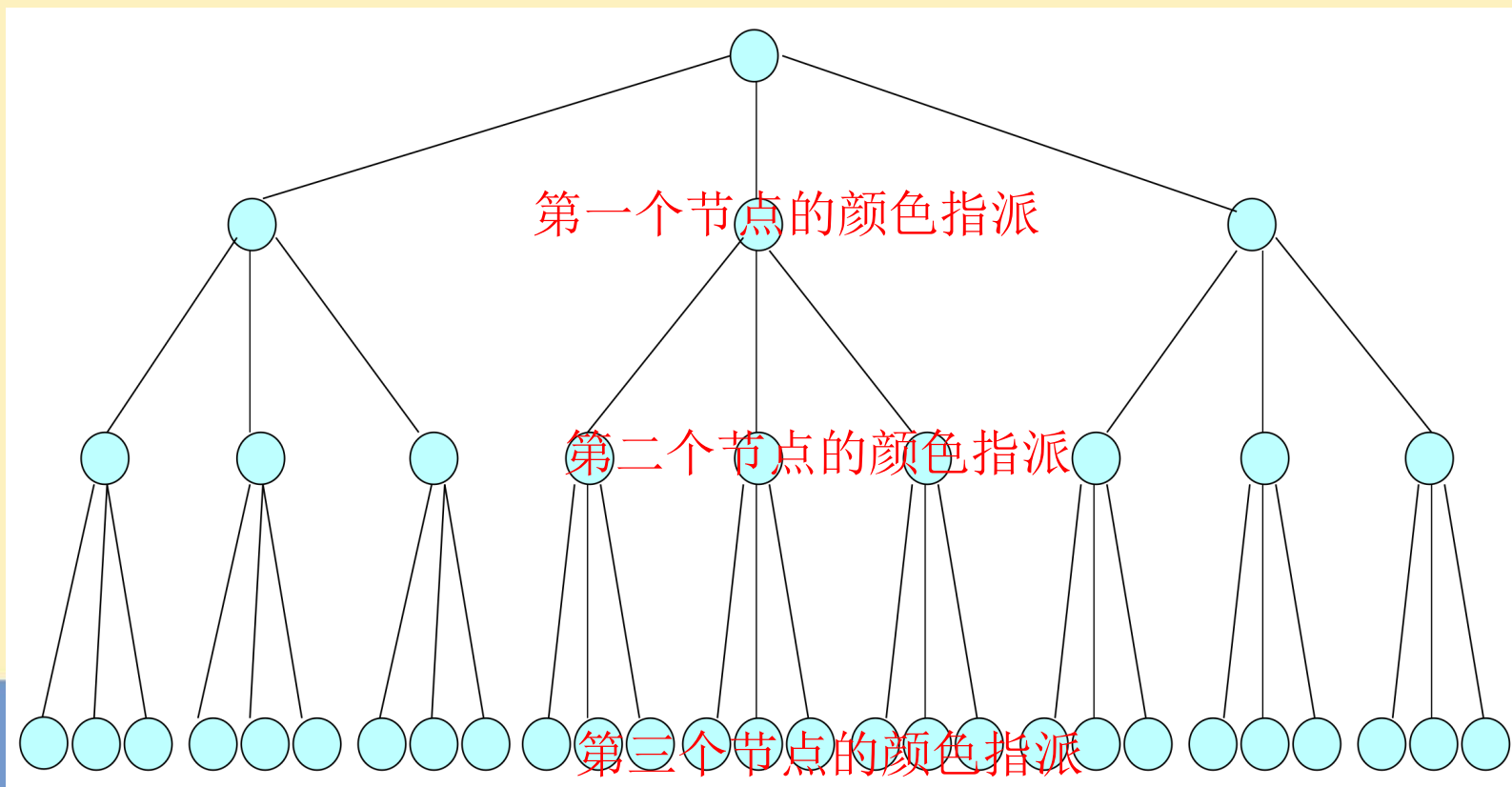
- 给出一个无向图 $G=(V,E)$ ，需要用三种颜色之一为 V 中的每个顶点着色（三种颜色编号分别为1, 2和3），使得没有两个邻接的顶点着同样的颜色。

解的表示

- 设图的顶点数为 n ，用 n 元组 $(c_1, c_2, \dots, c_n)$ 来表示顶点的一种着色方案， $c_i \in \{1, 2, 3\}$ ， $1 \leq i \leq n$ 。一个 n 个顶点的图共有 3^n 种可能的着色(合法的和非法的)。
- 例如， $(1,2,2,3,1)$ 表示一个有5个顶点的图的一种着色方案，一共有 3^5 种可能的着色(合法的和非法的)。

8.2.1 图的三着色问题

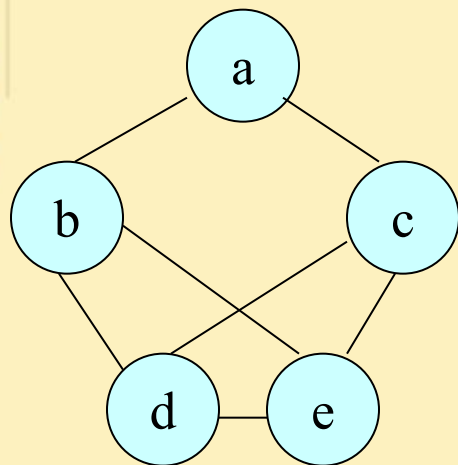
- 所有可能的着色方案的集合可以用一棵完全的三叉树来表示。在这棵树中，从根到叶节点的每一条路径代表一种着色指派。



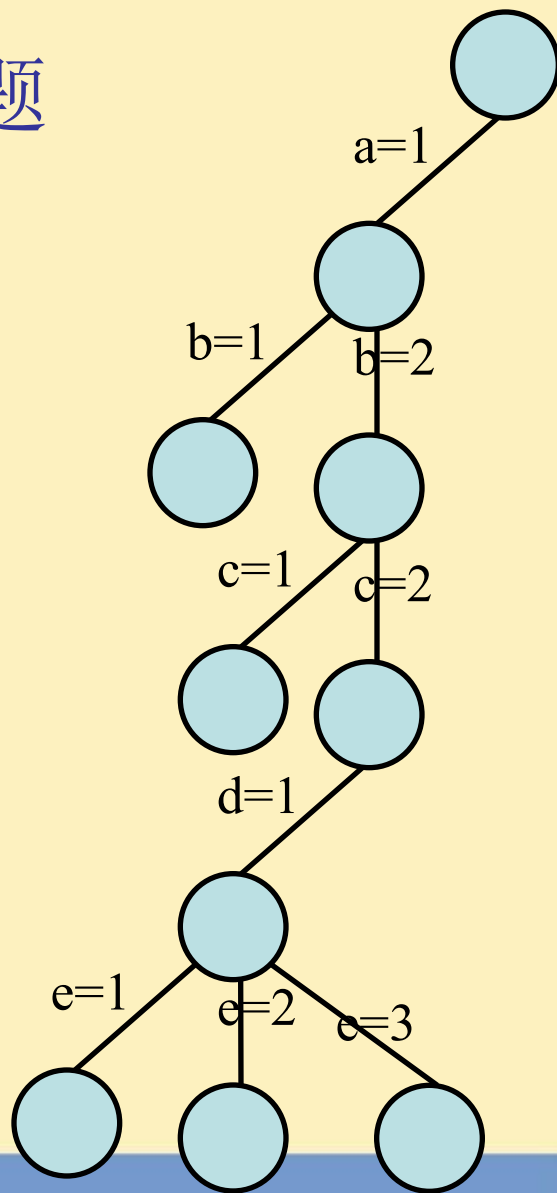
8.2.1 图的三着色问题

- 部分（可行）解——如果没有两个邻接的着色顶点有相同颜色，那么图的一个不完全着色称为**部分（可行）解**。
- 回溯思路：通过每次一个结点地生成隐含的树的方式来工作。
 - 如果从根到当前结点（叶**结点**）的路径对应于一个完整的合法着色，则过程就终止（除非要求找到所有可能的着色方案）；
 - 从根结点到当前**结点**的路径的长度小于 n 时，如果对应的着色是部分解，那么就生成当前**结点**的一个子**结点**，并将该子**结点**标记为**现结点**；如果对应的路径不是部分解，那么将**现结点**标记为**死结点**，并生成对应于另一种颜色的**新结点**。
 - 如果所有三种颜色都已经试过且没有成功，那么搜索就回溯到父结点，它的颜色被改变，依次类推。

8.2.1 图的三着色问题



Find a solution!



回溯法执行过程的观察

- 完整的树包含364个结点： $1+3+9+27+81+243=364$
- 但是，用回溯搜索方法仅生成了树中的10个结点后就得到了一个符合着色要求的完整解。实际搜索效率高。
- 结点是用深度优先搜索的方法生成的。
- 并不需要存储整棵树，只需要存储根到当前活结点的路径。事实上，根本没有生成有形的结点，整棵树是隐含的。在上述3着色的实例中，只需要保存颜色指派的踪迹就可以了。

算法 8.1 -COLORREC (递归)

输入: 无向图 $G=(V,E)$.

输出: G 的顶点的3着色 $c[1..n]$, 其中每个 $c[j]$ 为1,2或3。

1. **for** $k \leftarrow 1$ **to** n
2. $c[k] \leftarrow 0$
3. **end for**
4. $flag \leftarrow false$
5. $graphcolor(1)$
6. **if** $flag$ **then output** c
7. **else output** “no solution”

过程 $graphcolor(k)$

1. **for** $color=1$ **to** 3
2. $c[k] \leftarrow color$
3. **if** c 为合法着色 **then** set $flag \leftarrow true$ and **exit**
4. **else if** c 是部分解 **then** $graphcolor(k+1)$
5. **end for**

算法8.2-COLORITER (迭代)

输入: 无向图 $G=(V,E)$.

输出: G 的顶点的3着色 $c[1..n]$, 其中每个 $c[j]$ 为1,2或3。

```
1. for  $k \leftarrow 1$  to  $n$ 
2.    $c[k] \leftarrow 0$ 
3. end for
4.  $flag \leftarrow false$ 
5.  $k \leftarrow 1$ 
6. while  $k \geq 1$ 
7.   while  $c[k] \leq 2$ 
8.      $c[k] \leftarrow c[k] + 1$ 
9.     if  $c$ 为合法着色, then 设置 $flag \leftarrow true$ 
        且从两个while循环退出
10.    else if  $c$ 是部分解then  $k \leftarrow k + 1$  {前进}
11.  end while
12.   $c[k] \leftarrow 0$ 
13.   $k \leftarrow k - 1$  {回溯}
14. end while
15. if  $flag$  then output  $c$ 
16. else output "no solution"
```

算法的复杂度分析

- 对于这两种算法的时间复杂性，我们注意到在最坏情况下生成 $O(3^n)$ 个节点。对于每个生成的节点，如果当前着色是合法的、部分解，或者二者都不是，那么需要 $O(n)$ 的工作来检查。
- 因此，在最坏情况下，全部的运行时间是 $O(n3^n)$ 。
- 空间复杂度是 $\Theta(n)$ 。

8.2.2 n-皇后问题

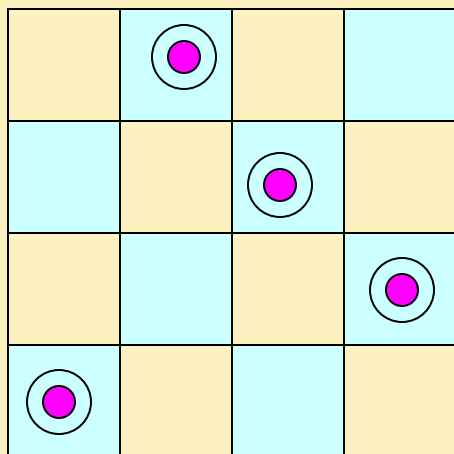
n-皇后问题描述:

- 如何在 $n \times n$ 的国际象棋棋盘上安排 n 个皇后, 使得没有两个皇后能互相攻击?
- 互相攻击: 如果两个皇后处在同一行、同一列或同一条对角线上, 则她们能互相攻击。
- 为了简化讨论, 研究4皇后问题。

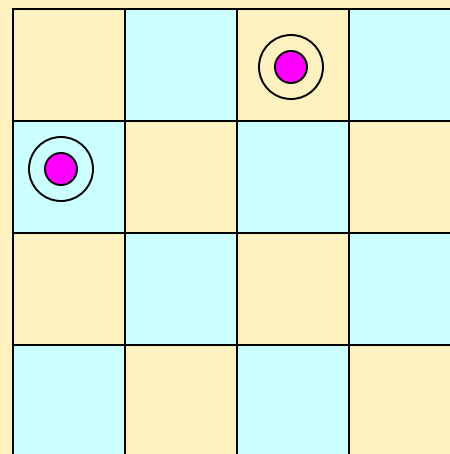
4-皇后问题

- 考虑一个4x4的棋盘，由于没有两个皇后能处在同一行，所以在解的表示上约定每个皇后都在不同的行上。
- 因此，用一个有4个分量的向量 $x=(x_1, x_2, x_3, x_4)$ 来描述4个皇后的一个可能布局，其中第 i 个分量 x_i 表示第 i 个皇后放在第 i 行的列位置。
- 又因为每行有4个位置，因此解空间一共有 4^4 种可能的布局。
- 实际上，由于没有两个皇后能放在同一列上，一个合法的放置对应于数1, 2, 3, 4的一个排列，这样解的搜索空间从 4^4 减少到 $4!$ 。

4-皇后问题的布局举例



The vector (2,3,4,1)



The partial vector (3,1)

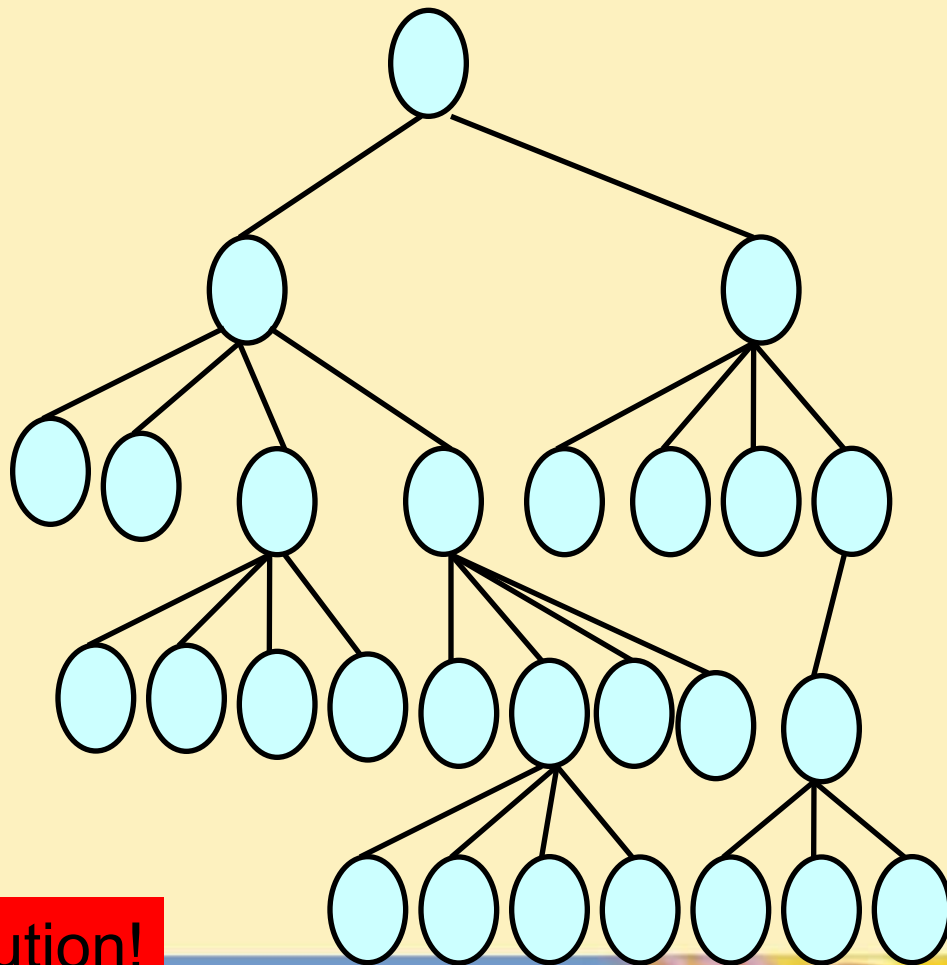
4-皇后问题的回溯求解过程

1			
X	X	2	
X	X	X	X

1			
			2
X	3		
X	X	X	X

1			
			2
		X	X

	1		
X	X	X	2
3			
X	X	4	



Find a solution!

互相攻击对应的情形

n-皇后问题的解由 $(x_1, \dots, x_n)$ 表示, x_i 表示第 i 个皇后放在第 i 行第 x_i 列。

- 因为每行只放一个皇后, 两个皇后不可能处在同一行上。(不存在行上攻击的情形)
- 如果两个不同的皇后 i 和 j 处在同一列上, 他们互相攻击: $x_i = x_j$;
- 如果两个不同的皇后 i 和 j 处在同一条对角线上, 他们互相攻击: $x_i - x_j = i - j$ or $x_i - x_j = j - i$.

算法8.3 4-QUEENS(迭代)

输入: 空。

输出: 对应于4皇后问题的解的向量 $x[1..4]$ 。

```
1.  for  $k \leftarrow 1$  to 4
2.       $x[k] \leftarrow 0$  {没有皇后放置在棋盘上}
3.  end for
4.  flag  $\leftarrow$  false
5.   $k \leftarrow 1$ 
6.  while  $k \geq 1$ 
7.      while  $x[k] \leq 3$ 
8.           $x[k] \leftarrow x[k] + 1$ 
9.          if  $x$  为合法解 then set flag  $\leftarrow$  true 且从两个while循环退出
10.         else if  $x$  是部分解 then  $k \leftarrow k + 1$  {前进}
11.     end while
12.      $x[k] \leftarrow 0$ 
13.      $k \leftarrow k - 1$  {回溯}
14. end while
15. if flag then output  $x$ 
16. else output “no solution”
```

算法的复杂度分析

- 现在考虑一个求解一般的 n 皇后问题的蛮力方法
 - 由于没有两个皇后能放在同一列上，则解向量必须是数 $1, 2, \dots, n$ 的一个**排列**。这样，蛮力方法可以改进为测试 $n!$ 种布局而不是 n^n 种。
 - 考虑 $(n-2)!$ 个向量，它们对应于前两个皇后放在第一列的那些布局，蛮力方法要盲目地测试所有这些向量，而在回溯法中用 $O(1)$ 次测试就可以避免这些测试。
 - 尽管回溯法在最坏情况下要用 $O(n^n)$ 时间来求解 n 皇后问题，根据经验它在有效性上远远超过蛮力方法的 $O(n!)$ 时间，作为它的可期望运行时间通常要**快得多**。例如上述4-皇后问题，生成了总共341个可能节点中的27个节点，就找到解。

基于排列树框架的求解

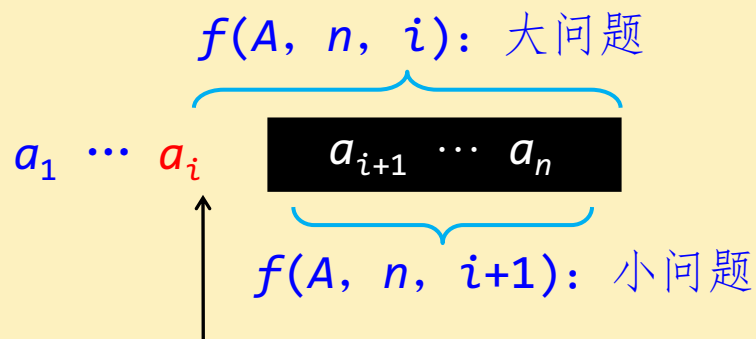
- 当求解问题是确定 n 个元素满足某种条件的排列时，相应的解空间树称为排列树。
- 既然4-皇后问题的解向量必须是一个排列，也可以直接基于排列树形式求解。
- 解空间为排列树的递归框架是以求全排列为基础的。

【例】有一个含 n 个整数的数组 $A[1..n]$ ，所有元素均不相同，求其所有元素的全排列。

例如， $A=\{1, 2, 3\}$ ，得到结果是 $(1, 2, 3)$ 、 $(1, 3, 2)$ 、 $(2, 3, 1)$ 、 $(2, 1, 3)$ 、 $(3, 1, 2)$ 、 $(3, 2, 1)$ 。

用数组A存放n个数的一个初始排列，采用递归法求解。

设 $f(A, n, i)$ 表示求 $A[i..n]$ （共 $n-i+1$ 个元素）的全排列，为大问题， $f(A, n, i+1)$ 表示求 $A[i+1..n]$ （共 $n-i$ 个元素）的全排列，为小问题。



a_i 位置取 $a_i \sim a_n$ 中每个元素，再组合 $f(A, n, i+1)$ 得到 $f(A, n, i)$

- a_i : a_i 与 a_i 交换, $f(A, n, i+1) \Rightarrow$ 以 a_i 开头的 $A[i..n]$ 的全排列
- a_{i+1} : a_i 与 a_{i+1} 交换, $f(A, n, i+1) \Rightarrow$ 以 a_{i+1} 开头的 $A[i..n]$ 的全排列
- ...
- a_n : a_i 与 a_n 交换, $f(A, n, i+1) \Rightarrow$ 以 a_n 开头的 $A[i..n]$ 的全排列

$f(A, n, i)$: 大问题

$a_1 \cdots a_i$

$a_{i+1} \cdots a_n$

$f(A, n, i+1)$: 小问题

a_i 位置取 $a_i \sim a_n$ 中每个元素, 再组合 $f(A, n, i+1)$ 得到 $f(A, n, i)$

求 A 的全排列的递归模型 $f(A, n, i)$

$f(A, n, i) \equiv$ 输出产生的解

当 $i > n$ 时

$f(A, n, i) \equiv$ 对于 $j=i \sim n$:

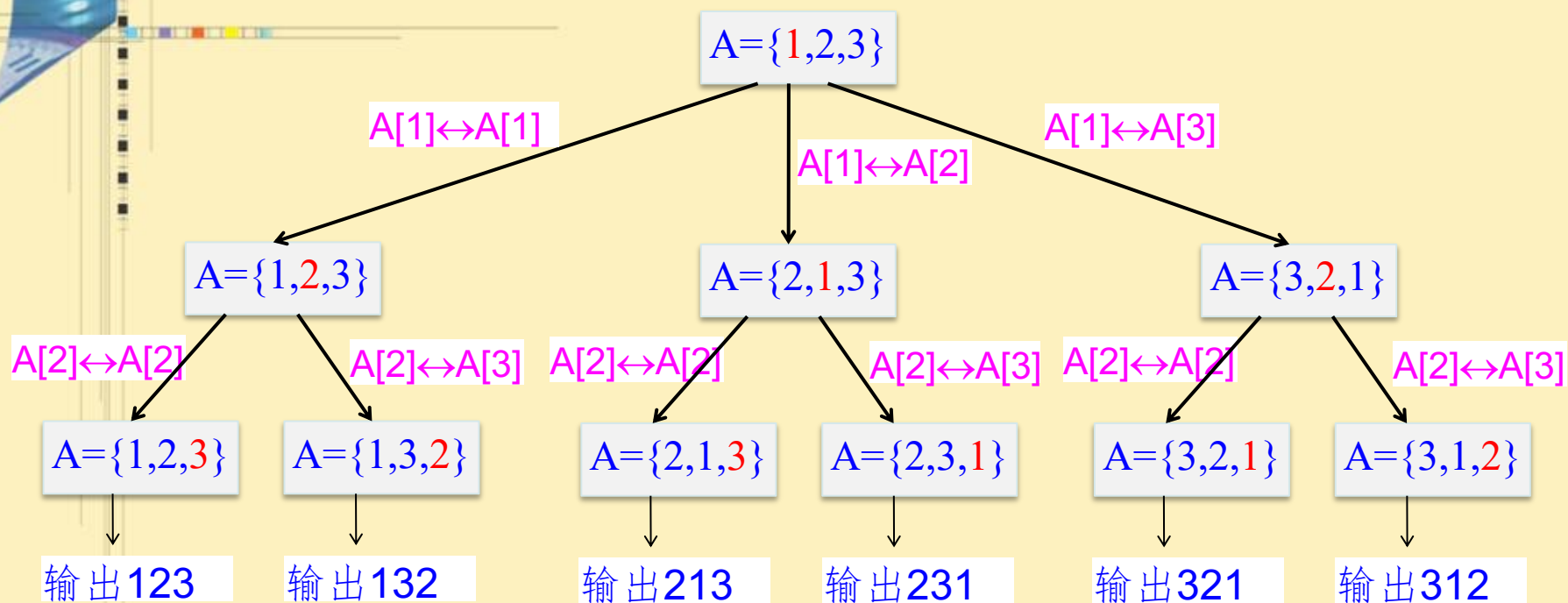
其他

$A[i]$ 与 $A[j]$ 交换位置;

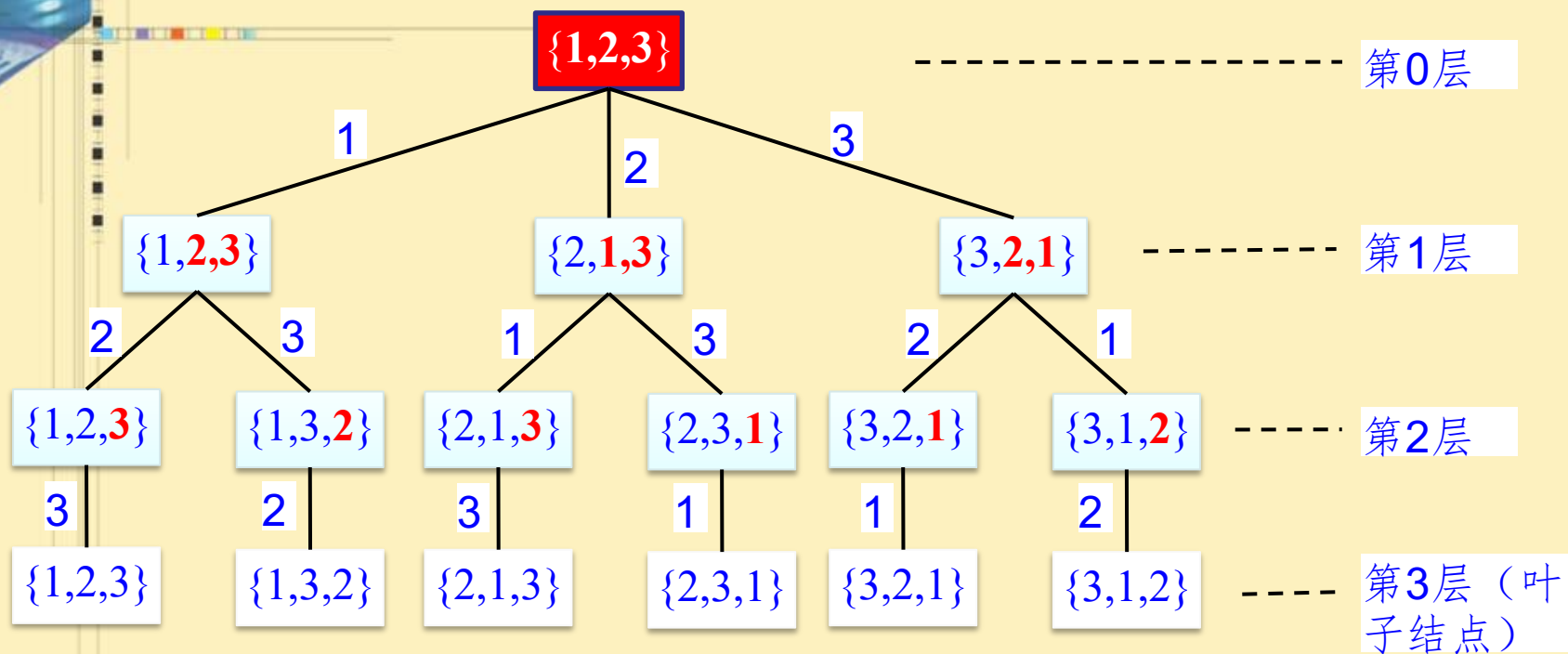
$f(A, n, i+1)$;

将 $A[i]$ 与 $A[j]$ 交换位置 (回溯)

求 $A=\{1, 2, 3\}$ 全排列



标准解空间



求数组 A （不含重复整数）的全排列递归算法

算法8.4 PERM-N

输入: $A[1..n]$ // n 个不重复的整数

输出: $A[1..n]$ 中 n 个数的全排列

1. perm(1)

过程 perm (i) { // 递归算法

1. if ($i > n$) then // 递归出口

2. 输出 $A[1..n]$ // 输出一个排列

3. else

4. for ($j \leftarrow i$ to n)

5. swap(i, j); // 交换 $A[i]$ 与 $A[j]$

6. perm ($i+1$);

7. swap(i, j); // 交换 $A[i]$ 与 $A[j]$: 恢复

8. end for

9. end if

算法8.5 4-QUEENS(排列树)

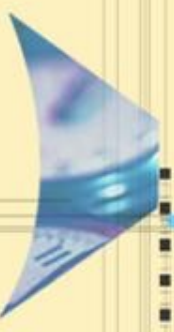
输入: 空。

输出: 对应于4皇后问题的解的向量 $x[1..4]$ 。

1. **for** $k \leftarrow 1$ **to** 4
2. $x[k] \leftarrow k$ // {皇后的初始摆放}
3. **end for**
4. $\text{flag} \leftarrow \text{false}$
5. $\text{queens}(1)$
6. **if** flag **then output** x
7. **else output** “no solution”

过程 $\text{queens}(i)$

1. **if** $i > 4$ **then**
2. $\text{flag} \leftarrow \text{true}; \text{exit}$ // 找到合法解, 返回
3. **else**
4. **for** $j \leftarrow 1$ **to** 4
5. $\text{swap}(i, j)$
6. **if** $x[1..i]$ 是合法的 **then**
7. $\text{queens}(i+1)$ {前进}
8. **end if**
9. $\text{swap}(i, j)$ {回溯}
10. **end for**
11. **end if**
12. **end if**
13. **end for**
14. **end if**



主要内容

8.1 引言

8.2 回溯法简单例

8.3 一般回溯法

8.4 回溯法的典型应用

回溯法对求解问题的要求

- 一般回溯算法，它可以作为一种系统的搜索方法应用到一类搜索问题当中。这类问题的解必须能表示成一个**元组**的形式，其中每个分量取自某个有穷且可枚举的集合。所有可能的元组（**也称向量**）构成可能的解空间。问题求解就是要从此解空间中搜索其中满足约束条件或者使某个目标函数取极值的一个向量，有时还要找出满足要求的所有向量。
- 约束条件分两种类型：**显式约束**和**隐式约束**。
 - **显式约束**限定每个分量只从一个给定的集合上取值。满足显式约束的所有元组确定一个可能的**解空间**；
 - **隐式约束**规定解空间中那些实际上满足要求的元组。因此，描述了分量之间必须彼此相关的情况。

回溯算法的适用条件-多米诺性质

设 $P(x_1, x_2, \dots, x_i)$ 为真表示向量 $\langle x_1, x_2, \dots, x_i \rangle$ 满足某个性质 (例如, n 皇后问题中前 i 个皇后放置在彼此不能攻击的位置)

多米诺性质:

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

要使回溯法得到正确应用, 性质 P 必须满足多米诺性质。

回溯算法的适用条件-多米诺性质

例：求不等式的所有整数解

$$5x_1 + 4x_2 - x_3 \leq 10, \quad 1 \leq x_i \leq 3, \quad i=1,2,3$$

$P(x_1, \dots, x_k)$ ：将 $x_1, x_2, \dots, x_k$ 代入原不等式的相应部分使得左边小于等于10。例如， $P(x_1, x_2, x_3)$ 表示 $5x_1 + 4x_2 - x_3 \leq 10$ 。

显然，存在解 $(x_1, x_2, x_3) = (1, 2, 3)$ ，使得 $P(x_1, x_2, x_3)$ 为真。但是， $P(x_1, x_2)$ 表示 $5x_1 + 4x_2 \leq 10$ ，显然为假。

因此，P不满足多米诺性质。不能用回溯法得到正确的解（因为回溯法搜索过程中会丢掉 $(1, 2, 3)$ 这个解）。

回溯算法的适用条件-多米诺性质

例：求不等式的所有整数解

$$5x_1 + 4x_2 - x_3 \leq 10, \quad 1 \leq x_i \leq 3, \quad i=1,2,3$$

将该问题进行如下变换：

令 $x_3 = 3 - x_3'$,

$$5x_1 + 4x_2 + x_3' \leq 13, \quad 1 \leq x_1, x_2 \leq 3, \quad 0 \leq x_3' \leq 2$$

显然，变换后的不等式满足多米诺性质，可以使用回溯法求解。对所得到的解很容易转换成原问题的解。

向量的长度

- 在前面问题例中，解采用固定长度的向量($x_1, x_2, \dots, x_i$)（这里 i 是0到 n 之间的某个整数，其中 n 是一个取决于问题规模的常量）。
 - 例如前面提到的3着色问题求解算法， i 就是图中节点数，固定不变；在皇后问题算法中， i 是皇后的个数，也是固定不变的。
- 但是对有些问题，解向量既可以采用固定长度的向量表示形式，也可以采用不定长的向量表示形式。不同的解的表示形式，搜索算法的设计细节不同。

例：子集和问题

子集和问题：给定一个含有 n 个整数的集合 $X = \{x_1, x_2, \dots, x_n\}$ 和一个整数 y ，找出和等于 y 的 X 的子集 Y 。

设 $X = \{10, 20, 30, 40, 80, 60\}$ ， $y = 60$ ，显然有3个子集合满足要求： $\{10, 20, 30\}$ ； $\{20, 40\}$ ； $\{60\}$ 。

- 如果采用变长的解表示形式，即直接用包含的元素本身构成子集，显然有三个不同长度的解：
 $\{10, 20, 30\}$ ； $\{20, 40\}$ ； $\{60\}$
- 如果采用定长的解表示形式，就是设计一个长度为 n 的0-1向量来表示 X 的一个子集，0和1分别代表 X 中对应的元素不出现或者出现在子集合中， n 为 X 中元素的个数，则三个解的长度相同，分别为：
 $\{1, 1, 1, 0, 0, 0\}$ ； $\{0, 1, 0, 1, 0, 0\}$ ； $\{0, 0, 0, 0, 0, 1\}$ 。

回溯法的本质——约束穷举

假设对某个问题的解用一个定长的 n -元组 $(x_1, x_2, \dots, x_n)$ 表示其解， x_i 的取值集合 X_i 的大小为 m_i ，规范函数为 $P(x_1, x_2, \dots, x_n)$ （约束函数或者优化目标函数）。令 $m = m_1 m_2 \cdots m_n$ 。

- 硬性处理：通过穷举方式求解方法，需要构造 m 个 n -元组并逐一测试它们是否满足 $P(x_1, x_2, \dots, x_n)$ ，从而找出该问题的解。
- 回溯处理：不断地用规范函数 $P_i(x_1, x_2, \dots, x_n)$ （ P_i 可能是根据搜索进程不断对 P 的修改，有时称为**限界函数**）去测试正在构造中的 n -元组的部分向量 $(x_1, x_2, \dots, x_i)$ ，看其是否可能导致（最优）解。如果判定 $(x_1, x_2, \dots, x_i)$ 不可能导致（最优）解，那么就将可能要测试的 $m_{i+1} m_{i+2} \cdots m_n$ 个向量一概略去。
- 回溯法比硬性处理作的测试（ m 次）要少得多。

一般回溯法

- 在回溯过程中，解向量中每个 x_i 都属于一个有限的线序集 X_i ；因此，回溯算法按词典序考虑笛卡儿积 $X_1 \times X_2 \times \cdots \times X_n$ 的元素。
- 算法最初从空向量开始，然后选择 X_1 中最小元素作为 x_1 ，如果 (x_1) 是一个部分解，算法通过从 X_2 中选择最小的元素作为 x_2 继续，如果 (x_1, x_2) 是一个部分解，那么就包括 X_3 中最小的元素，否则 x_2 被置为 X_2 中的下一个元素。
- 一般地，假定算法已经检测到部分解为 $(x_1, x_2, \cdots, x_j)$ ，然后再去考虑向量 $v = (x_1, x_2, \cdots, x_j, x_{j+1})$ 。有下面几种情况：

一般回溯法 (continue)

1. 如果 v 表示问题的最后解，算法记录下它作为一个解，在仅希望获得一个解时终止，或者继续去找出其他解。
2. (**向前步骤**)。如果 v 表示一个部分解，算法通过选择集合 X_{j+2} 中的最小元素前进。
3. 如果 v 既不是最终的解，也不是部分解，则有两种子情况：
 - a) 如果从集合 X_{j+1} 中还有其他的元素可选择，算法将 x_{j+1} 置为 X_{j+1} 中的下一个元素。
 - b) (**回溯步骤**)。如果集合 X_{j+1} 中没有更多的元素可选择，算法通过将 x_j 置为 X_j 中的下一个元素而回溯；如果集合 X_j 中仍然没有其他的元素可以选择，算法通过将 x_{j-1} 置为 X_{j-1} 中的下一个元素而回溯，依次类推。

算法 BACKTRACKITER

输入: 集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述。

输出: 解向量 $v=(x_1, x_2, \dots, x_i)$, $0 \leq i \leq n$.

```
1.  $v \leftarrow ()$ 
2.  $\text{flag} \leftarrow \text{false}$ 
3.  $k \leftarrow 1$ 
4. while  $k \geq 1$ 
5.     while  $X_k$  没有被穷举
6.          $x_k \leftarrow X_k$  中的下一个元素; 将  $x_k$  加入  $v$ 
7.         if  $v$  为最终解 then set  $\text{flag} \leftarrow \text{true}$ 
            且两个while循环退出
8.         else if  $v$  是部分解 then  $k \leftarrow k+1$  {前进}
9.     end while
10.    重置 $X_k$ , 使得下一个元素排在第一位
11.     $k \leftarrow k-1$ 
12. end while
13. if  $\text{flag}$  then output  $v$ 
14. else output “no solution”
```

算法 BACKTRACKREC

输入:集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述。

输出:解向量 $v=(x_1, x_2, \dots, x_i)$, $0 \leq i \leq n$.

1. $v \leftarrow ()$
2. $\text{flag} \leftarrow \text{false}$
3. $\text{advance}(1)$
4. **if** flag **then** output v
5. **else** output “no solution”

过程 $\text{advance}(k)$

1. **for** each $x \in X_k$
2. $x_k \leftarrow x$; 将 x_k 加入 v
3. **if** v 为最终解 **then** set $\text{flag} \leftarrow \text{true}$ and **exit**
4. **else if** v 是部分解 **then** $\text{advance}(k+1)$
5. **end for**

算法 BACKTRACK // 基于排列树的递归框架

输入: 问题的输入

输出: 解向量 $x[1..n]$ 。

```
1.  for  $k \leftarrow 1$  to  $n$ 
2.       $x[k] \leftarrow k$  // {初始排列}
3.  end for
4.  flag  $\leftarrow$  false
6.  advance (1)
7.  if flag then output  $x$ 
8.  else output “no solution”
```

过程 advance (i)

```
1.  if ( $i > n$ ) then                                // 求解排列树的递归框架
2.      flag  $\leftarrow$  true; exit                    // 搜索到叶子结点, 输出一个可行解
3.  else
4.      for ( $j \leftarrow i$  to  $n$ )                    // 用j枚举 $x[i]$ 所有可能候选值
5.          swap( $x[i], x[j]$ );                        // 通过交换来实现
6.          if legal(i) then                          // 是否满足约束条件和限界条件
7.              advance (i+1);                        // 满足条件进入下一层
8.          end if
9.          swap( $x[i], x[j]$ );                        // 恢复状态: 回溯
10. end for
11. end if
```

